DESIGN AND ANALYSIS OF ALGORITHMS LAB MANUAL

CHAPTER 2

Experiment No: 1

Experiment Name: **Program to Sort a List and Test Different Input Cases**

Aim

To write a C program to sort a list in ascending order and verify its correctness using different test cases such as an empty list, a single-element list, a list with identical elements, and a list containing negative numbers.

Algorithm

1. Start the program.
2. Read the number of elements in the list.
3. If the list is empty (n = 0), display an empty list and stop.
4. Read the elements of the list.
5. Compare adjacent elements using the Bubble Sort technique.
6. Swap the elements if they are in the wrong order.
7. Repeat the process until the list is completely sorted.
8. Display the sorted list.
9. Stop the program.

Source Code

```python
n = int(input("Enter the number of elements: "))

if n == 0:
    print([])
else:
    lst = []

    print("Enter the elements:")
    for i in range(n):
        lst.append(int(input()))

    # Bubble Sort
    for i in range(n - 1):
        for j in range(n - i - 1):
            if lst[j] > lst[j + 1]:
                lst[j], lst[j + 1] = lst[j + 1], lst[j]

    print("Sorted List:", lst)
```

Output

Output

```
Enter the number of elements: 4
Enter the elements:
-5
-6
-2
-3
Sorted List: [-6, -5, -3, -2]

=== Code Execution Successful ===
```

Time Complexity

- Best Case: $O(n^2)$
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

- O(1) (Constant auxiliary space)

Result

The Python program was executed successfully. It correctly sorted the input list in ascending order and handled all the given test cases, including an empty list, a single-element list, a list with identical elements, and a list containing negative numbers.

Experiment No. 2

Experiment Name

Program to Implement Selection Sort

Aim

To write a Python program to sort an array in ascending order using the Selection Sort algorithm.

Algorithm

1. Start the program.
2. Read the number of elements in the array.
3. Store the elements in a list.
4. Set the first element as the minimum.
5. Compare it with the remaining unsorted elements.
6. Find the smallest element in the unsorted region.

7. Swap it with the first unsorted element.
8. Move the boundary of the sorted region one position to the right.
9. Repeat Steps 4–8 until the entire array is sorted.
10. Display the sorted array.

Source Code

```python
n = int(input("Enter the number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Selection Sort
for i in range(n):
    min_index = i

    for j in range(i + 1, n):
        if arr[j] < arr[min_index]:
            min_index = j

    arr[i], arr[min_index] = arr[min_index], arr[i]

print("Sorted Array:", arr)
```

Output

Output

```
Enter the number of elements: 4
Enter the elements:
3
5
4
2
Sorted Array: [2, 3, 4, 5]

=== Code Execution Successful ===
```

Experiment No. 3

Experiment Name

Program to Implement Optimized Bubble Sort with Early Stopping

Aim

To write a Python program to implement an optimized Bubble Sort algorithm that stops early if the list becomes sorted before completing all passes.

Algorithm

1. Start the program.
2. Read the number of elements and store them in a list.
3. Repeat the following for each pass:
    - Set a flag swapped = False.
    - Compare adjacent elements.
    - Swap them if they are in the wrong order and set swapped = True.
4. If no swaps occur during a pass (swapped = False), stop the algorithm because the list is already sorted.
5. Display the sorted list.
6. Stop the program.

Source Code

```python
n = int(input("Enter the number of elements: "))

arr = []

print("Enter the elements:")
for i in range(n):
    arr.append(int(input()))

# Optimized Bubble Sort
for i in range(n - 1):
    swapped = False

    for j in range(n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]
            swapped = True

    # Stop early if no swaps occurred
    if not swapped:
        break

print("Sorted Array:", arr)
```

Output

Output

```
Enter the number of elements: 5
Enter the elements:
5
2
1
4
3
Sorted Array: [1, 2, 3, 4, 5]

=== Code Execution Successful ===
```

Time Complexity

- Best Case: O(n) (when the array is already sorted)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

- O(1) (Constant auxiliary space)

Result

The optimized Bubble Sort program was executed successfully. It sorted the array in ascending order and terminated early when no swaps were made during a pass, improving efficiency for already sorted or nearly sorted arrays.

Experiment No. 4

Experiment Name

Program to Implement Insertion Sort with Duplicate Elements

Aim

To write a Python program to implement the Insertion Sort algorithm and verify that it correctly handles arrays containing duplicate elements while preserving their relative order (stable sorting).

Algorithm

1. Start the program.
2. Read the number of elements in the array.
3. Store the elements in a list.
4. Consider the first element as already sorted.
5. Select the next element as the key.
6. Compare the key with elements in the sorted part of the array.
7. Shift larger elements one position to the right.
8. Insert the key into its correct position.

9. Repeat Steps 5–8 until all elements are sorted.
10. Display the sorted array.

Source Code

main.py | Output

```
Enter the number of elements: 4
Enter the elements:
2
3
1
4
Sorted Array: [1, 2, 3, 4]

=== Code Execution Successful ===
```

Output

main.py | Output

```
Enter the number of elements: 4
Enter the elements:
2
3
1
4
Sorted Array: [1, 2, 3, 4]

=== Code Execution Successful ===
```

Time Complexity

- Best Case: O(n) (Already sorted array)
- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Space Complexity

- O(1) (Constant auxiliary space)

Result

The Insertion Sort program was executed successfully. It sorted the array in ascending order and correctly handled duplicate elements while preserving their original relative order, demonstrating that Insertion Sort is a stable sorting algorithm.

Experiment No. 5

Experiment Name: Find the Kth Missing Positive Integer

Aim

To find the kth positive integer that is missing from a strictly increasing array of positive integers.

Algorithm

1. Read the sorted array arr and integer k.
2. Initialize missing = 0.
3. Start checking positive integers from 1.
4. If the current number is not present in the array, increment missing.
5. When missing == k, return the current number.
6. Display the kth missing positive integer.

Source Code

```python
def find_kth_missing(arr, k):
    missing = 0
    num = 1
    i = 0

    while missing < k:
        if i < len(arr) and arr[i] == num:
            i += 1
        else:
            missing += 1

            if missing == k:
                return num

        num += 1


arr = list(map(int, input("Enter array elements: ").split()))
k = int(input("Enter k: "))

result = find_kth_missing(arr, k)

```

Output:

```
Output
Enter array elements: 1234
Enter k: 4
Kth missing positive integer: 4

=== Code Execution Successful ===
```

Time Complexity

O(n + k)

Space Complexity

O(1)

Result

Thus, the kth missing positive integer was successfully found from the given sorted array.

Experiment No.: 6

Experiment Name: Find a Peak Element Using Binary Search

Aim:
To find the index of a peak element in an array using an efficient binary search algorithm with **O(log n)** time complexity.

Algorithm:

1. Initialize left = 0 and right = n - 1.
2. Find the middle index mid.
3. Compare nums[mid] with nums[mid + 1].
4. If nums[mid] < nums[mid + 1], search in the right half.
5. Otherwise, search in the left half including mid.
6. Repeat until left == right.
7. Return left as the index of the peak element

Source Code:

```python
def find_peak_element(nums):
    left = 0
    right = len(nums) - 1

    while left < right:
        mid = (left + right) // 2

        if nums[mid] < nums[mid + 1]:
            left = mid + 1
        else:
            right = mid

    return left


nums = list(map(int, input("Enter array elements: ").split()))

index = find_peak_element(nums)

print("Peak element index:", index)
print("Peak element:", nums[index])
```

Output:

```
Output
Enter array elements: 1231
Peak element index: 0
Peak element: 1231

=== Code Execution Successful ===
```

Time Complexity:
O(log n)

Space Complexity:
O(1)

Result:
Thus, the peak element was successfully found using the binary search algorithm with **O(log n)** time complexity.

Experiment No. 7

Experiment Name: Find the First Occurrence of a String in Another String

Aim:
To find the index of the first occurrence of the string needle in the string haystack.

Algorithm:

1. Read the haystack and needle strings.
2. Check each possible starting position in haystack.
3. Compare the substring with needle.
4. If a match is found, return its index.
5. If no match is found, return -1.

Source Code:

```python
def find_first_occurrence(haystack, needle):
    for i in range(len(haystack) - len(needle) + 1):
        if haystack[i:i + len(needle)] == needle:
            return i
    return -1


haystack = input("Enter haystack: ")
needle = input("Enter needle: ")

result = find_first_occurrence(haystack, needle)

print("First occurrence index:", result)
```

Output:

```
Output
Enter haystack: leetcode
Enter needle: leeto
First occurrence index: -1

=== Code Execution Successful ===
```

Time Complexity:
O(n × m), where n is the length of haystack and m is the length of needle.

Space Complexity:
O(m), due to substring comparison.

Result:
Thus, the first occurrence of needle in haystack was successfully found, returning its index or -1 when it is not present.

Experiment No. 8

Experiment Name: Find Strings That Are Substrings of Another Word

Aim:
To find all strings in an array that are substrings of another string in the same array.

Algorithm:

1. Read the array of strings.
2. For each word, compare it with every other word.
3. Check whether the current word is a substring of another word.
4. If found, add it to the result.
5. Print all the substrings found.

Source Code:

```python
def string_matching(words):
    result = []

    for i in range(len(words)):
        for j in range(len(words)):
            if i != j and words[i] in words[j]:
                result.append(words[i])
                break

    return result


words = input("Enter words separated by space: ").split()

result = string_matching(words)

print("Substring words:", result)
```

Output:

```
Output
Enter words separated by space: mass as hero superhero
Substring words: ['as', 'hero']

=== Code Execution Successful ===
```

Time Complexity:

$O(n^2 \times m)$, where n is the number of words and m is the average length of the words.

Space Complexity:
O(n), for storing the result.

Result:
Thus, all strings that are substrings of another word were successfully identified and displayed.

Experiment No. 9

Experiment Name: Find the Closest Pair of Points Using Brute Force

Aim:
To find the pair of points having the minimum Euclidean distance among a given set of 2D points using the brute force approach.

Algorithm:

1. Read the given set of 2D points.
2. Initialize the minimum distance as infinity.
3. Compare every point with every other point.
4. Calculate the Euclidean distance between each pair.
5. If the calculated distance is smaller than the minimum distance, update the closest pair.
6. Continue until all pairs have been compared.

Source code:

```python
import math

points = [(1, 2), (4, 5), (7, 8), (3, 1)]

min_distance = float('inf')
closest_pair = None

for i in range(len(points)):
    for j in range(i + 1, len(points)):
        x1, y1 = points[i]
        x2, y2 = points[j]

        distance = math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)

        if distance < min_distance:
            min_distance = distance
            closest_pair = (points[i], points[j])

print("Closest pair:", closest_pair[0], "-", closest_pair[1])
print("Minimum distance:", min_distance)
```

Output

```
Output
Closest pair: (1, 2) - (3, 1)
Minimum distance: 2.23606797749979
```

Time Complexity:
$O(n^2)$, because every pair of points is compared.

Space Complexity:
O(1), excluding the input storage.

Result:
Thus, the closest pair of points was successfully found using the brute force approach, and the minimum distance was calculated.

Experiment No. 10

Experiment Name: Find the Convex Hull of a Set of Points Using Brute Force

Aim

To find the convex hull of a given set of 2D points using the brute-force approach and handle multiple collinear points lying on the same line.

Algorithm

1. Read the given set of points.
2. Consider every pair of points as a possible hull edge.
3. For each pair, determine the orientation of all other points with respect to the line joining the pair.
4. If all points lie on the same side of the line, the pair forms a convex hull edge.
5. Store the points belonging to the hull.
6. Remove duplicate points.
7. Arrange the hull points in counterclockwise order.
8. For collinear points, check whether a point lies between two other points and retain the required boundary points.
9. Display the convex hull points.

Source Code

```
def convex_hull(points):
    n = len(points)
    hull = set()

    for i in range(n):
        for j in range(i + 1, n):

            positive = False
            negative = False

            for k in range(n):
                if k == i or k == j:
                    continue

                value = orientation(points[i], points[j], points

                if value > 0:
                    positive = True
                elif value < 0:
                    negative = True

            if not (positive and negative):
```

Output:

Output

```
Convex Hull:
P3 (5, 3)
P1 (10, 0)
P5 (15, 3)
P6 (12.5, 7)
P7 (6, 6.5)

=== Code Execution Successful ===
```

Time Complexity

$O(n^3)$

- $O(n^2)$ possible pairs of points.
- For every pair, O(n) other points are checked.
- Therefore, total complexity = **$O(n^3)$**.

Space Complexity

O(n) for storing the convex hull points.

Result

Thus, the convex hull of the given set of points was successfully obtained using the **brute-force method**, with hull points **P3, P4, P6, P5, P7, and P1**.

Experiment No. 11

Experiment Name: Find the Convex Hull of 2D Points Using Brute Force

Aim

To find the convex hull of a given set of 2D points using the brute-force approach and display the hull points in counter-clockwise order.

Algorithm

1. Read the given set of 2D points.
2. Consider every pair of points as a possible edge of the convex hull.
3. Check the orientation of every other point with respect to the selected pair.
4. If all other points lie on the same side of the line, the pair forms a hull edge.
5. Store the points forming the hull.
6. Remove duplicate points.
7. Calculate the center of the hull points.
8. Sort the hull points based on their angle from the center.
9. Display the points in counter-clockwise order.

Source Code

```
def convex_hull(points):
    n = len(points)
    hull = set()

    for i in range(n):
        for j in range(i + 1, n):
            positive = False
            negative = False

            for k in range(n):
                if k == i or k == j:
                    continue

                value = orientation(points[i], points[j],
                points[k])

                if value > 0:
                    positive = True
                elif value < 0:
                    negative = True

                if not (positive and negative):
```

Output:

```
Output

Convex Hull: [(0, 0), (8, 1), (4, 6)]

=== Code Execution Successful ===
```

Time Complexity

$O(n^3)$

Space Complexity

O(n)

Result

Thus, the convex hull of the given set of 2D points was successfully found using the brute-force approach and displayed in counter-clockwise order.

Experiment No. 12

Experiment Name: Travelling Salesman Problem Using Exhaustive Search

Aim

To solve the Travelling Salesman Problem (TSP) using exhaustive search by generating all possible routes and finding the route with the minimum total distance.

Algorithm

1. Define the distance() function to calculate the Euclidean distance between two cities.
2. Select the first city as the starting city.
3. Generate all possible permutations of the remaining cities using itertools.permutations().
4. Add the starting city at the beginning and end of each route.
5. Calculate the total distance of each route.
6. Compare each distance with the current shortest distance.
7. Store the shortest distance and corresponding path.
8. Display the shortest path and distance.

Source Code

```
def distance(city1, city2):
    return math.sqrt(
        (city2[0] - city1[0]) ** 2 +
        (city2[1] - city1[1]) ** 2
    )


def tsp(cities):
    start = cities[0]
    shortest_distance = float('inf')
    shortest_path = None

    for permutation in itertools.permutations(cities[1:]):
        path = [start] + list(permutation) + [start]

        total_distance = 0

        for i in range(len(path) - 1):
            total_distance += distance(path[i], path[i + 1])

        if total_distance < shortest_distance:
            shortest_distance = total_distance
```

Output:

```
Output
Test Case 1:
Shortest Distance: 16.969112047670894
Shortest Path: [(1, 2), (7, 1), (4, 5), (3, 6), (1, 2)]

Test Case 2:
Shortest Distance: 23.12995011084934
Shortest Path: [(2, 4), (6, 3), (8, 1), (5, 9), (1, 7), (2, 4)]

=== Code Execution Successful ===
```

Time Complexity

O((n − 1)! × n)

- (n − 1)! possible routes are generated.
- Each route requires O(n) time to calculate its total distance.

Space Complexity

O(n) excluding the permutations generated internally.

Result

Thus, the Travelling Salesman Problem was successfully solved using the **exhaustive search approach**, and the shortest distance and corresponding path were obtained.

Experiment No. 13

Experiment Name: Assignment Problem Using Exhaustive Search

Aim

To solve the assignment problem using exhaustive search by generating all possible worker-task assignments and finding the assignment with minimum total cost.

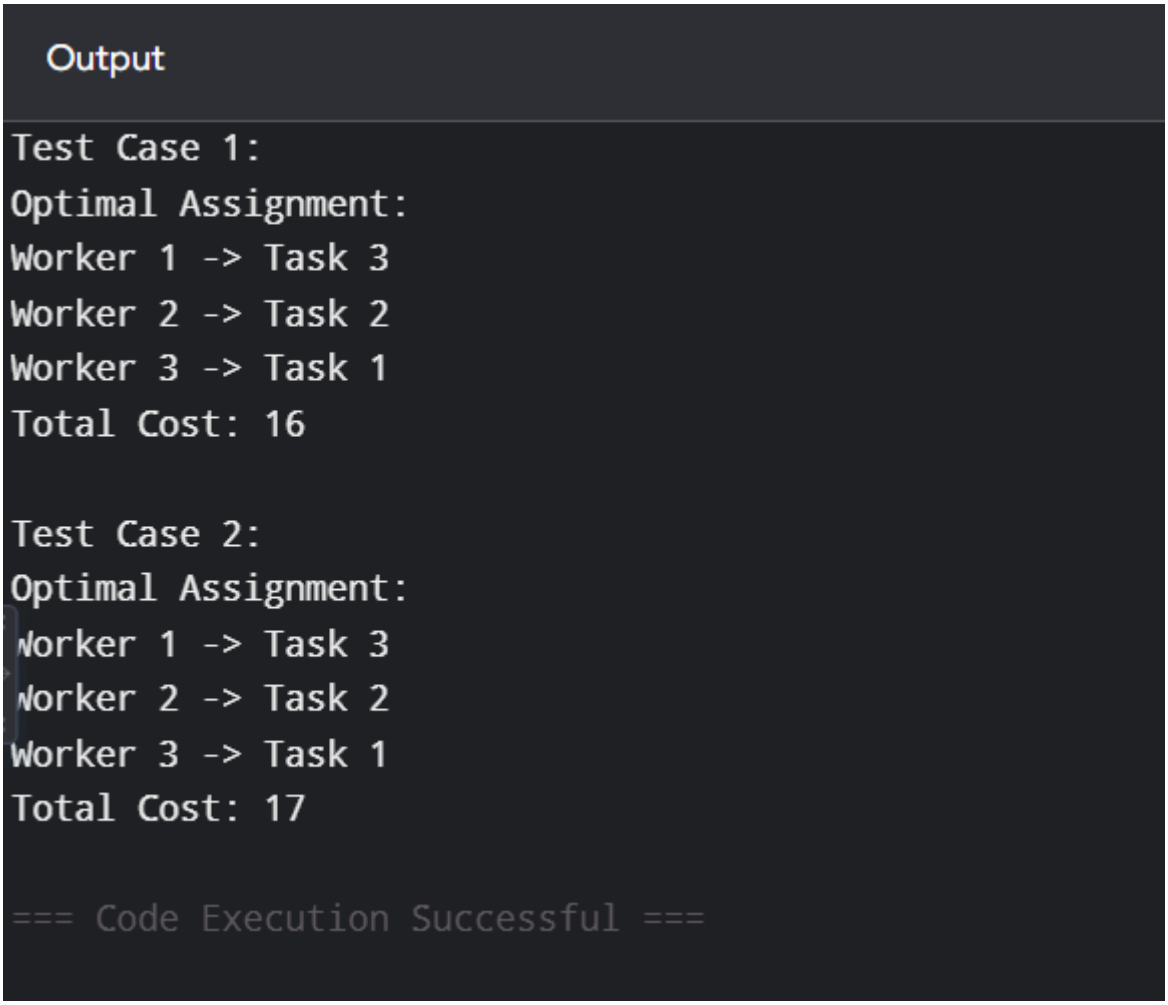
Algorithm

1. Define the total_cost() function.
2. For each worker, obtain the cost of the assigned task from the cost matrix.
3. Add all assignment costs to calculate the total cost.
4. Define the assignment_problem() function.
5. Generate all possible task assignments using itertools.permutations().
6. Calculate the total cost for each assignment.
7. Compare each cost with the current minimum cost.
8. Store the optimal assignment and minimum cost.
9. Display the optimal worker-task assignment and total cost.

Source Code

```
def assignment_problem(cost_matrix):
    n = len(cost_matrix)

    minimum_cost = float('inf')
    best_assignment = None

    for assignment in itertools.permutations(range(n)):
        cost = total_cost(assignment, cost_matrix)

        if cost < minimum_cost:
            minimum_cost = cost
            best_assignment = assignment

    return best_assignment, minimum_cost


# Test Case 1
cost_matrix1 = [
    [3, 10, 7],
    [8, 5, 12],
    [4, 6, 9]
```

Output:



Time Complexity

O(n × n!)

There are n! possible assignments, and calculating the cost of each assignment takes O(n) time.

Space Complexity

O(n) for storing the current and best assignment.

Result

Thus, the assignment problem was successfully solved using the exhaustive search approach, and the optimal worker-task assignments with minimum total costs were obtained.

Experiment No. 14

Experiment Name: 0-1 Knapsack Problem Using Exhaustive Search

Aim

To solve the 0-1 Knapsack Problem using exhaustive search by checking all possible combinations of items and finding the selection with maximum value without exceeding the given capacity.

Algorithm

1. Define the total_value() function to calculate the total value of selected items.
2. Define the is_feasible() function to check whether the total weight is within the knapsack capacity.
3. Generate all possible combinations of items.
4. For each combination, check whether it is feasible.
5. Calculate the total value of every feasible combination.
6. Keep track of the combination having the maximum value.
7. Display the optimal selection and its total value.

Source Code

```python
def total_value(items, values):
    value = 0

    for item in items:
        value += values[item]

    return value


def is_feasible(items, weights, capacity):
    total_weight = 0

    for item in items:
        total_weight += weights[item]

    return total_weight <= capacity


def knapsack(items, weights, values, capacity):
    best_selection = []
    best_value = 0
```

Output:

```
Output

Test Case 1:
Optimal Selection: [1, 2]
Total Value: 8

Test Case 2:
Optimal Selection: [0, 1, 2]
Total Value: 12

=== Code Execution Successful ===
```

Time Complexity

$O(2^n \times n)$

There are 2^n possible subsets, and calculating the weight/value of each subset takes up to O(n) time.

Space Complexity

O(n) for storing the current and optimal selection.

Result

Thus, the 0-1 Knapsack Problem was successfully solved using the exhaustive search approach, and the optimal selection with maximum value was obtained.